

Programmeringsparadigmer 2025

Session 4: Funktioner og lister

Hans Hüttel

30. september 2025

Vores plan for i dag

1. Læringsmålene
2. Præsentation af forberedelsesopgaverne
3. Opgave nr. 1
4. Diskussion: Gift ved første blik
5. Pause
6. Opgave nr. 2
7. Diskussion: Det dumme kolon
8. Opgave nr. 3
9. Opgave nr. 4
10. Opgave nr. 5
11. Hvis tiden tillader det: Flere opgaver i eget tempo.
12. Vi evaluerer dagens session – **bliv venligst til slut!**

Læringsmål

- ▶ At forstå syntaksen og den uformelle semantik for betingede udtryk og guards og kunne anvende disse ved programmering i Haskell
- ▶ At forstå og kunne bruge de forskellige former for mønstre og mønstermatchning ved programmering i Haskell
- ▶ At forstå anonyme funktioner (lambda-udtryk) i Haskell
- ▶ At forstå begrebet listeabstraktion og hvordan guards bruges i listeabstraktioner
- ▶ At kunne bruge listeabstraktion til at definere funktioner i Haskell
- ▶ At forstå, hvordan alle disse begreber kan bruges, når man bygger større programmer, og kunne anvende denne abstraktion ved programmering i Haskell

Forberedelsesopgave – onlytwo

Definer, ved hjælp af mønstermatchning og *uden at bruge længdefunktionen*, en funktion `onlytwo`, der fortæller os, om en liste har præcis to elementer – i hvilket tilfælde den skal returnere `True` – eller ej, i hvilket tilfælde den skal returnere `False`. Hvad er typen af `onlytwo`?

Forberedelsesopgave – alldots

Prikproduktet af to talpar (a, b) og (c, d) er tallet $a \cdot c + b \cdot d$.
Definer, *ved brug af listeabstraktion*, en funktion `alldots`, der tager to lister af talpar og returnerer alle mulige prikprodukter af hvert par fra den første liste og hvert par fra den anden liste. Find to gode testcases til at afprøve din funktion og brug dem til at teste din kode. Hvad er typen af `alldots`?

Opgave 1 – idhead (10 minutter)

Definer en funktion `idhead`, der fortæller os, om en liste af par starter med et par, hvis første og andet element er identiske. Vi ønsker, at

```
idhead [(42,42) ,(3,4) ,(484000,5)]
```

giver os `True`, men at

```
idhead [( " plip" ,"mango" ) ,( " dingo" ," kpst" ) ]
```

giver os `False`. I din definition må du *ikke* bruge `head` eller `if-then-else`-udtryk! Er funktionen polymorf? Hvis ja, på hvilken måde?

Diskussion – Gift ved første blik

Under pauserne i optagelserne til *Gift ved første blik* kom et af parrene op at skændes om datastrukturer i Haskell. Her er, hvad de sagde:

- ”Liste” er bare et kluntet ord for array! Begge er lange datastrukturer, og vi bruger firkantede parenteser til at definere begge. Hutton er et fjols. Han ved ikke, hvad han taler om, og han kender ikke almindelig terminologi.
- Årh altså! Der må da være en grund til, at man kalder dem lister og ikke arrays. Jeg vil skilles!
- Åh ja?! Giv mig bare *én god grund* til, at lister ikke er arrays! Jeg bliver den første af os to, der bliver skilt!!

Parret begyndte at skændes højlydt, og filmholdet måtte gribe ind for at give en korrekt forklaring. Hvad sagde filmholdet til dem?

Pause

Opgave 2 – Pyth (15 minutter)

En *pythagoræisk trippel* er en trippel (a, b, c) af naturlige tal a , b , og c , således at $a \leq b < c$ og $a^2 + b^2 = c^2$. Med andre ord, en trippel af denne form giver os længderne af de tre sider i en retvinklet trekant, hvor alle sider har heltal-længde. Den mindste pythagoræiske trippel er $(3, 4, 5)$.

Brug listeabstraktion til at definere en funktion `pyt`, der, når den får et heltal k , giver os en liste af alle pythagoræiske tripler, hvis største element er højst k . Før du skriver definitionen af `pyt`, skal du finde ud af, hvad dens type bør være.

Opgave 3 – Bighead (15 minutter)

Sidste uge læste vi, at en berømt influencer på Instagram har defineret en Haskell-funktion `bighead`, der kan fortælle os, hvor mange elementer i en liste `xs` er større end ($>$) det første element i `xs`. Som eksempel på funktionens opførsel vil resultatet af `bighead [7,4,5,8,9]` være `2`. Nu er det din tur til at være en berømt influencer. Hvordan ville du definere `bighead`-funktionen? Hvad bør dens type være?

Diskussion – Det dumme kolon

En berømt influencer har lagt en video op på TikTok om liste-operationer i Haskell. Videoen fik 3 millioner likes inden for de første 24 timer. I videoen siger influenceren:

Jeg undrer mig over, hvorfor nogen overhovedet ville bruge det mærkelige kolon-tegn (det har ikke engang et navn!!). Det kan kun bruges i mønstre til at skrive `x:xs` i en funktionsdefinition.

Hvis vi vil sætte et element `x` foran en liste `xs`, så er vi nødt til at skrive `[x] ++ xs`.

*Haskell er et dumt og kluntet sprog! Designerne af Haskell er **inkompetente** og **pinlige**, og hvis de har et job (hvilket jeg tvivler på), så skal de **fyres** og **dræbes**. Det hele er deres skyld.*

Har influenceren ret?

Opgave 4 – En portion curry (10 minutter)

Vis, hvordan meningen med følgende curriede funktionsdefinition kan udtrykkes ved hjælp af lambda-udtryk i Haskell.

```
plonk x y z = x+y+z
```

Find frem til typen af `plonk` uden at spørge Haskell-fortolkeren.

Opgave 5 – At være perfekt (10 minutter)

Et perfekt tal n er et naturligt tal, der er summen af sine egne divisorer, når n selv udelades. 28 er et perfekt tal, da $1 + 2 + 4 + 7 + 14 = 28$. Brug listeabstraktion til at definere en funktion `isperfect`, der fortæller os, om et givet naturligt tal er et perfekt tal.

Evaluering

- ▶ Hvad var svært?
- ▶ Hvad overraskede jer?
- ▶ Hvad gik godt?